

BITS Academic Course Recommender

Postman Round 2

1 Objective

Build an **agentic AI-based BITS Academic Course Recommender** that helps a BITS student choose courses for a semester. The system must use the **provided BITS academic data** as its primary source of truth and follow the academic requirements and policies applicable to the student's programme, batch, and academic history.

The system should first determine what the student is required and eligible to take, and only then recommend courses according to the student's interests and natural-language preferences.

2 Input Data

The project will be supplied with:

- BITS Academic Regulations,
- BITS Bulletin,
- current-semester timetable,
- course handouts for the available courses, and
- a student profile / academic history.

Use the supplied data directly. Do not invent course rules, course properties, or academic requirements that cannot be supported by the provided material.

3 Data Pre-processing and Structure

The supplied documents contain a large amount of material that is not useful for course recommendation. The raw documents should therefore be **pre-processed into a clean structured dataset before being used by the recommender**.

The recommendation system should retrieve from this structured representation rather than repeatedly passing complete PDFs or large unprocessed text chunks to the model.

A possible structure is shown below.

Entity	Suggested fields
Student	campus, batch, degree / dual degree, current semester, completed courses, current courses, minor, interests
Course	course code, title, department, units, category, topics, prerequisites, restrictions
Programme Rules	CDC / DEL / HUEL / OPEL requirements, programme-specific and batch-specific rules
Handout Data	attendance policy, midsem, compre, evaluation components, makeup policy, instructor, syllabus
Timetable	course code, section, instructor, days, hours, room, midsem slot, compre slot
Source Metadata	source document, page / section reference, extraction confidence where required

The pre-processing stage should remove irrelevant material, normalize course codes and categories, and store the useful academic information in a consistent schema. If an item cannot be extracted reliably, it should be marked for verification instead of being guessed.

4 Student Profile

The dashboard should allow a student to create and update a profile containing at least:

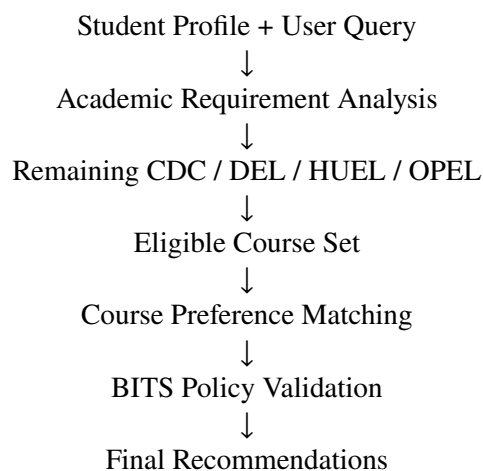
- campus and admission year,
- degree / dual degree,
- current semester,
- completed and current courses,
- minor, if applicable, and
- academic interests.

The profile is used to determine the student's academic state before any recommendation is made.

5 Course Recommendation Behaviour

The system must first calculate the student's remaining academic requirements. For example, it should identify remaining CDC, DEL, HUEL and OPEL requirements and apply the relevant prerequisites, restrictions and programme rules.

The core flow should be:



The system should not recommend a course simply because its description matches the user's interests. A recommendation must first be academically valid for that student's profile.

6 Natural-Language Queries

The student should be able to ask questions directly through the dashboard, for example:

“Suggest DELs related to AI.”

“I want an OPEL with no attendance requirement.”

“Suggest courses with no midsem and a lenient makeup policy.”

“I need a HUEL and prefer project-based evaluation.”

The system should translate the query into preferences, retrieve relevant eligible courses from the structured dataset, and explain why each recommendation satisfies the request.

7 Course Handout Use

Course handouts should support preference-based recommendations. The system may use structured handout information such as attendance rules, evaluation pattern, midsem / compre presence, projects, quizzes, makeup policy, instructor and course topics.

If a requested property is not explicitly available in the supplied data, the answer should state that it could not be verified.

8 Brownie Point: Timetable Intelligence

Timetable intelligence is an optional advanced feature. The system can use the current-semester timetable to ensure that suggested courses form a feasible schedule.

It may check class, tutorial, lab, midsem and comprehensive-examination clashes and may also support preferences such as:

- no 8 AM classes,
- keeping a particular weekday free,
- avoiding long gaps between classes,
- preferring a compact timetable, and
- selecting a suitable section when multiple sections are available.

For example, if one section clashes with a CDC but another section is clash-free, the system should prefer the feasible section rather than rejecting the course immediately.

9 Implementation Guidance

- Pre-process the supplied documents into structured records before building the recommendation layer.
- Keep academic-rule checking deterministic wherever possible; use the LLM mainly for user-intent understanding, semantic interest matching and explanation.
- Keep source references with extracted academic facts so recommendations can be traced back to the supplied documents.
- Validate extracted course codes, prerequisites, categories and programme requirements before using them.
- Design ingestion so that a new semester timetable or a new set of handouts can be added without changing the recommendation logic.
- Keep the final recommendation concise: requirement satisfied, eligibility, relevant course properties, and why it matches the user’s request.

10 Final Deliverables

1. Working Dashboard

A working dashboard where a student can create or update a profile and ask natural-language course-selection queries such as “Suggest an AI-related DEL with no midsem” or “Find an OPEL with no attendance requirement.” The dashboard should calculate the student’s applicable requirements, retrieve eligible courses, and generate recommendations **live from the processed data**. Course lists, eligibility checks, remaining requirements, and answers should not be hardcoded.

2. Git Repository

A clean Git repository containing the application code together with the **data pre-processing and data-retrieval pipeline**. It should include the scripts or modules used to parse the supplied documents, build the structured data, retrieve relevant academic information at runtime, and run the dashboard. Include clear setup and run instructions in the README.